NAME : B.Chaithanya Prakash Reddy

REG NO : 192425255

Q6. Debugging Bubble Sort Code

The following Bubble Sort implementation does not correctly sort the array and performs unnecessary iterations. Identify logical errors in loop conditions and swapping mechanism, and explain their causes. Debug the code step-by-step to ensure correct sorting. Optimize the algorithm by reducing unnecessary passes using a flag. Analyze the time complexity of the corrected version. Rewrite the final optimized code with proper comments.

def bubble_sort(arr):

n = len(arr)

for i in range(n):

for j in range(n-1):

if arr[j] < arr[j+1]: # ERROR

arr[j], arr[j+1] = arr[j+1], arr[j]

return arr

Error 1: Wrong Comparison Operator

if arr[j] < arr[j+1]:

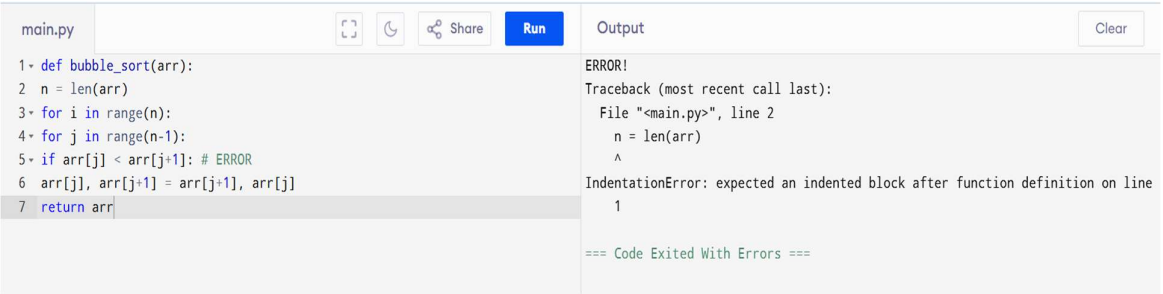
Correction:

if arr[j] > arr[j+1]:

Problem:

- This swaps elements when the left element is **smaller** than the right.

- It sorts the array in **descending order** instead of ascending order.



Error 2: Unnecessary Inner Loop Iterations

for j in range(n-1):

Problem:

- After every pass, the largest element reaches its correct position.
- Comparing it again is unnecessary.

Correction:

for j in range(n-i-1):

This reduces unnecessary comparisons.

Error 3: No Early Stopping

The algorithm continues even if the array is already sorted.

Solution :
Use a **flag**.

If no swaps occur in one pass:

swapped = False

After swapping:

swapped = True

If

if not swapped:

    break

the algorithm stops early.

2. Step-by-Step Debugging

Example:

arr = [5, 2, 4, 1]

Pass 1

Compare 5 and 2

Swap

[2,5,4,1]

Compare 5 and 4

Swap

[2,4,5,1]

Compare 5 and 1

Swap

[2,4,1,5]

Largest element **5** is fixed.

Pass 2

Compare 2 and 4

No swap

[2,4,1,5]

Compare 4 and 1

Swap

[2,1,4,5]

Largest unsorted element **4** is fixed.

Pass 3

Compare 2 and 1

Swap

[1,2,4,5]

Array becomes sorted.

Pass 4

No swaps occur.

Algorithm stops because of the flag.

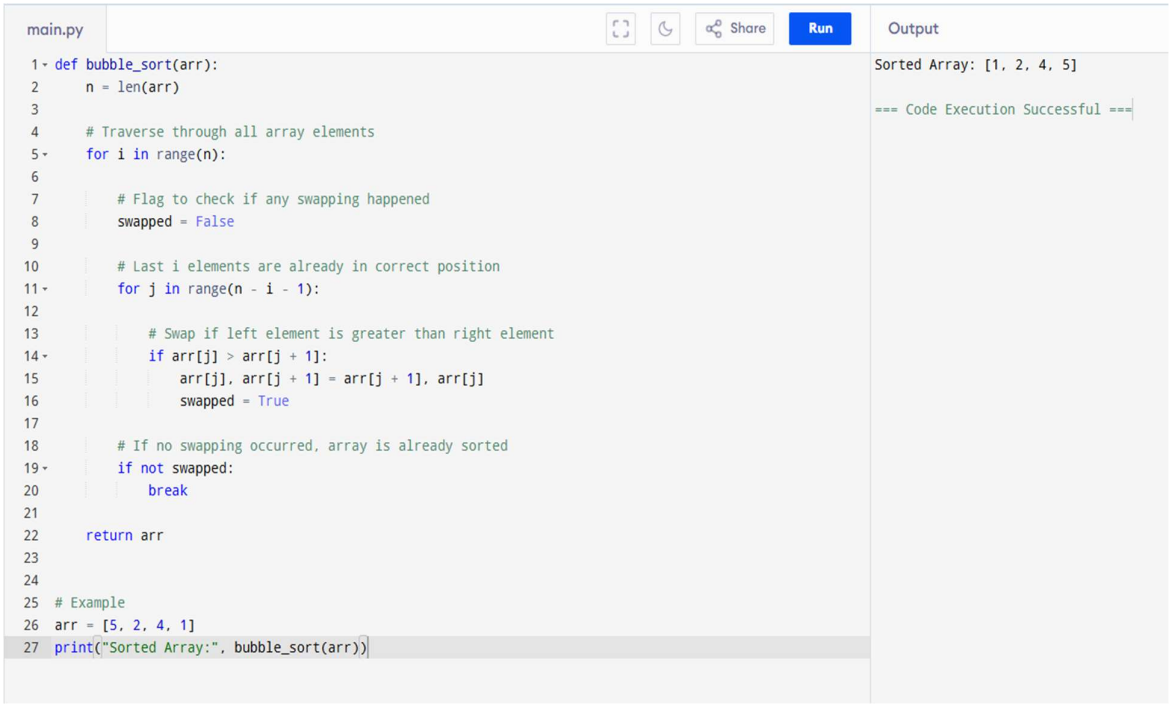
3. Optimized Bubble Sort Code

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Flag to check if any swapping happened
        swapped = False
        # Last i elements are already in correct position
        for j in range(n - i - 1):
            # Swap if left element is greater than right element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no swapping occurred, array is already sorted
        if not swapped:
```

```
            break
    return arr
# Example
arr = [5, 2, 4, 1]
print("Sorted Array:", bubble_sort(arr))
```

Output :

Sorted Array: [1, 2, 4, 5]



5. Time Complexity Analysis

Case	Time Complexity
Best Case (already sorted)	**O(n)**
Average Case	**O(n²)**

Case	Time Complexity
Worst Case (reverse sorted)	**$O(n^2)$**

Space Complexity

O(1) (In-place sorting)

Optimization Summary

- Corrected comparison from < to > for ascending order.
- Reduced unnecessary comparisons using range(n - i - 1).
- Added a swapped flag to stop early when the array is already sorted.
- Improved best-case performance from **$O(n^2)$** to **O(n)** while keeping the average and worst cases at **$O(n^2)$**.

Algorithm: Optimized Bubble Sort

1. **Start**.
2. Read the input array arr of size n.
3. Repeat for each pass from i = 0 to n-1:
    - Initialize a Boolean variable swapped = False.
4. Compare each pair of adjacent elements from index 0 to n-i-2.
5. If arr[j] > arr[j+1], swap the two elements.
6. After swapping, set swapped = True.
7. Continue until the end of the current pass.
8. If swapped == False, terminate the algorithm because the array is already sorted.

9. Otherwise, continue with the next pass.
10. Repeat until all passes are completed or the array becomes sorted.
11. Display the sorted array.
12. **Stop**.